

طراحی و پیاده‌سازی سیستم آرکید وب‌محور بازی‌لند: معماری CGI، مدیریت Stateless و تجربه کاربری نوستالژیک

علیرضا زارع‌بیدکی

Alireza ZareBidoki

alireza@zarebidoki.com

<https://github.com/alireza-zarebidoki>

۶ دی ۱۴۰۴ — December 26, 2025

چکیده

این مقاله به معرفی و تحلیل معماری سیستم آرکید بازی‌لند (BAZI LAND Arcade System) می‌پردازد که یک پلتفرم وب‌محور برای اجرای بازی‌های آرکید کلاسیک با رویکرد نوستالژیک است. این سیستم شامل ۷ بازی تعاملی (حدس عدد، دوز محدود، سنگ کاغذ قیچی پنج حرکتی، حلقه آویز، فیوناچی ۲۰۴۸، شطرنج تغییر یافته، و چکرز) است که با استفاده از معماری CGI در زبان C پیاده‌سازی شده و با رابط کاربری JavaScript خالص و CSS3 با افکت‌های سه‌بعدی و نئونی ارائه می‌شود. سیستم دارای معماری stateless، سیستم احراز هویت مبتنی بر سشن، سیستم اقتصادی سکه‌محور، و پنل مدیریتی است. در این مقاله به تحلیل معماری، تصمیمات طراحی، جزئیات پیاده‌سازی، چالش‌ها، و راهکارهای آینده پرداخته می‌شود. کلمات کلیدی: سیستم آرکید، CGI، معماری Stateless، Vanilla JavaScript، احراز هویت، طراحی نوستالژیک، بازی‌های وب

Abstract:

This paper presents BAZI LAND, a web-based arcade gaming platform featuring 7 interactive games implemented using CGI architecture in C, with a nostalgic user interface built with vanilla JavaScript and CSS3 including 3D and neon effects. The system employs stateless architecture, session-based authentication, coin-based economy, and administrative management panel. We analyze the architecture, design decisions, implementation details, challenges, and future directions.

فهرست مطالب

۵	۱	مقدمه
۵	۱.۱	نگاهی کلی
۵	۲.۱	انگیزه و اهداف
۵	۳.۱	دامنه کاربرد
۶	۲	پیشینه و مفاهیم
۶	۱.۲	سیستم‌های آرکید کلاسیک
۶	۲.۲	معماری (Common Gateway Interface (CGI
۶	۳.۲	معماری Stateless
۷	۳	معماری سیستم
۷	۱.۳	نمای کلی معماری
۸	۲.۳	ساختار پروژه
۹	۳.۳	جریان اجرا
۹	۴	بازی‌های موجود
۱۰	۱.۴	حدس عدد (Guess Number)
۱۰	۲.۴	دوز محدود (Tic-Tac-Toe Limited)
۱۰	۳.۴	سنگ کاغذ قیچی++ (RPS 5-Move)
۱۱	۴.۴	حلقه آویز (Hangman Battle)
۱۱	۵.۴	فیوناچی ۲۰۴۸
۱۱	۶.۴	شطرنج تغییر یافته
۱۲	۷.۴	چکرز (Checkers)
۱۲	۵	جزئیات پیاده‌سازی
۱۲	۱.۵	Backend: توابع کمکی CGI
۱۲	۱.۱.۵	تابع print_header()
۱۲	۲.۱.۵	تابع get_param()
۱۳	۳.۱.۵	تابع url_decode()
۱۳	۲.۵	Frontend: معماری JavaScript
۱۳	۱.۲.۵	کاروسل بازی‌ها
۱۴	۲.۲.۵	ماژول احراز هویت

۳.۵	Frontend: افکت‌های بصری CSS	۱۵
۱.۳.۵	افکت نئونی (Neon Glow)	۱۵
۲.۳.۵	افکت سه‌بعدی (3D Perspective)	۱۵
۳.۳.۵	افکت CRT	۱۵
۴.۵	لایه داده: JSON Files	۱۶
۱.۴.۵	ساختار users.json	۱۶
۲.۴.۵	ساختار admin_logs.json	۱۶
۶	تصمیمات طراحی	۱۷
۱.۶	انتخاب C برای Backend	۱۷
۲.۶	انتخاب CGI به جای FastCGI	۱۷
۳.۶	انتخاب Vanilla JavaScript	۱۸
۴.۶	انتخاب فایل‌های JSON به جای پایگاه داده	۱۸
۷	چالش‌ها و محدودیت‌ها	۱۹
۱.۷	چالش‌های فنی	۱۹
۱.۱.۷	مدیریت State در CGI	۱۹
۲.۱.۷	Race Condition در نوشتن فایل‌های JSON	۱۹
۲.۷	محدودیت‌های امنیتی	۱۹
۱.۲.۷	رمزهای عبور Plain Text	۱۹
۲.۲.۷	عدم HTTPS	۱۹
۳.۲.۷	عدم CSRF Protection	۲۰
۸	تست و اعتبارسنجی	۲۰
۱.۸	تست دستی	۲۰
۲.۸	تست CGI	۲۰
۳.۸	محدودیت‌های تست	۲۰
۹	کارهای آینده	۲۰
۱.۹	بهبودهای فنی	۲۰
۱.۱.۹	انتقال به پایگاه داده	۲۰
۲.۱.۹	استفاده از WebSocket	۲۱
۳.۱.۹	امنیت	۲۱
۲.۹	فیچرهای جدید	۲۱
۱.۲.۹	بازی‌های جدید	۲۱

۲۲	Leaderboard	۲.۲.۹
۲۲	سیستم دستاوردها (Achievements)	۳.۲.۹
۲۲	بهبادهای UI/UX	۳.۹
۲۲	نتیجه‌گیری	۱۰
۲۲	دستاوردها	۱.۱۰
۲۳	درس‌های آموخته شده	۲.۱۰
۲۳	معماری CGI	۱.۲.۱۰
۲۳	State Management	۲.۲.۱۰
۲۳	امنیت	۳.۲.۱۰
۲۳	کاربردهای آموزشی	۳.۱۰
۲۴	جمع‌بندی نهایی	۴.۱۰
۲۵	نصب و راه‌اندازی	آ
۲۵	پیش‌نیازها	۱.آ
۲۵	کامپایل	۲.آ
۲۵	اجرا	۳.آ

۱ مقدمه

۱.۱ نگاهی کلی

بازی‌لند یک سیستم آرکید وب‌محور است که با هدف بازآفرینی تجربه کابینت‌های آرکیدی دهه ۱۹۸۰ و ۱۹۹۰ میلادی در محیط مرورگر طراحی و پیاده‌سازی شده است. این سیستم ترکیبی از تکنولوژی‌های سنتی (CGI with C) و مدرن (HTML5, CSS3, Vanilla JavaScript) را برای ایجاد یک تجربه یکپارچه و نوستالژیک به کار می‌گیرد.

۲.۱ انگیزه و اهداف

اهداف اصلی این پروژه عبارتند از:

- بازآفرینی تجربه آرکید کلاسیک: شبیه‌سازی کامل یک کابینت آرکید از جمله ظاهر بصری، سیستم سکه، و رابط کاربری مشابه.
- یادگیری معماری CGI: بررسی و پیاده‌سازی معماری Common Gateway Interface که یکی از اولین و بنیادی‌ترین روش‌های ایجاد وب اپلیکیشن‌های پویا است.
- طراحی Stateless: استفاده از معماری بدون وضعیت که در آن تمام state بازی در URL parameters انتقال می‌یابد.
- توسعه بدون Framework: استفاده از Vanilla JavaScript به جای کتابخانه‌های سنگین مانند React یا Vue برای درک عمیق‌تر مفاهیم بنیادی وب.
- تجربه کاربری منحصر به فرد: ایجاد یک رابط کاربری با افکت‌های نئونی، سه‌بعدی، و انیمیشن‌های پیچیده که حس استفاده از یک دستگاه آرکید واقعی را منتقل کند.

۳.۱ دامنه کاربرد

این سیستم برای موارد زیر قابل استفاده است:

- سرگرمی و بازی‌های کوتاه‌مدت
- آموزش برنامه‌نویسی وب و معماری CGI
- مطالعه موردی در طراحی سیستم‌های stateless
- پلتفرم نمونه برای توسعه‌دهندگان

۲ پیشینه و مفاهیم

۱.۲ سیستم‌های آرکید کلاسیک

دستگاه‌های آرکید در دهه‌های ۱۹۷۰ تا ۱۹۹۰ میلادی نقش مهمی در صنعت بازی داشتند. این دستگاه‌ها معمولاً دارای یک کابینت چوبی یا فلزی، مانیتور CRT، جویستیک، دکمه‌های رنگی، و سیستم سکه بودند. بازی‌لند قصد دارد تمام این عناصر را در یک محیط وب شبیه‌سازی کند.

۲.۲ معماری Common Gateway Interface (CGI)

CGI یک استاندارد برای اجرای برنامه‌های خارجی توسط سرور وب است که در سال ۱۹۹۳ معرفی شد. در این معماری:

۱. مرورگر یک درخواست HTTP به سرور ارسال می‌کند
۲. سرور یک process جدید برای اجرای برنامه CGI ایجاد می‌کند
۳. برنامه CGI خروجی HTML تولید می‌کند
۴. سرور خروجی را به مرورگر برمی‌گرداند
۵. Process پایان می‌یابد

مزایای CGI:

- سادگی و عدم نیاز به runtime سنگین
- هر درخواست مستقل است
- امکان استفاده از هر زبان برنامه‌نویسی

معایب CGI:

- ایجاد process جدید برای هر درخواست (overhead)
- عدم امکان حفظ state بین درخواست‌ها
- مقیاس‌پذیری محدود برای traffic بالا

۳.۲ معماری Stateless

در معماری stateless، سرور هیچ اطلاعاتی از وضعیت قبلی کلاینت را حفظ نمی‌کند. تمام اطلاعات لازم برای پردازش درخواست باید در خود درخواست موجود باشد. در بازی‌لند، تمام وضعیت بازی در URL parameters قرار می‌گیرد.

مثال:

/cgi-bin/tictactoe_limited.cgi?p1=0,4,8&p2=1,5&turn=1&move=2

در این URL:

● p1=0,4,8: مهره‌های بازیکن ۱ در خانه‌های ۰، ۴، ۸

● p2=1,5: مهره‌های بازیکن ۲ در خانه‌های ۱، ۵

● turn=1: نوبت بازیکن ۱

● move=2: حرکت بعدی به خانه ۲

۳ معماری سیستم

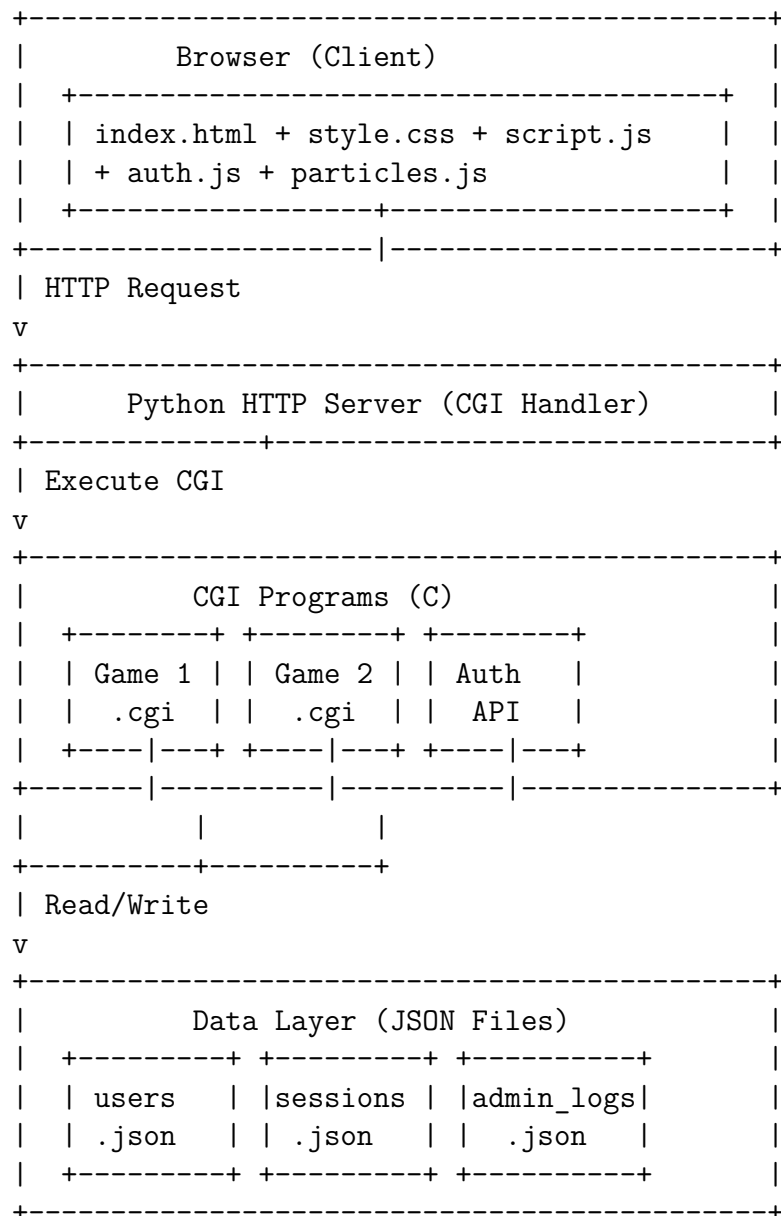
۱.۳ نمای کلی معماری

سیستم بازی‌لند از معماری سه‌لایه استفاده می‌کند:

۱. لایه رابط کاربری: (Frontend) HTML5, CSS3, Vanilla JavaScript

۲. لایه منطق: (Backend) برنامه‌های CGI نوشته شده با C

۳. لایه داده (Data): (Layer) فایل‌های JSON برای ذخیره‌سازی



شکل ۱: نمای کلی معماری سیستم بازی‌لند

۲.۳ ساختار پروژه

ساختار پوشه‌های پروژه به شرح زیر است:

```

1 test-project/
2   index.html      # Main page
3   style.css       # Styles (1128 lines)
4   script.js       # Frontend logic (1353 lines)
5   auth.js         # Auth module (328 lines)
6   Makefile        # Build system
7
8   src/
9     common/
10      c_utils.h    # CGI helper functions

```



```
11         c_utils.c
12
13     games/
14         guess_number.c
15         tictactoe_limited.c
16         rps_5move.c
17         hangman_battle.c
18         fibonacci_2048.c
19         chess/           # Chess (4 files)
20         checkers/        # Checkers (4 files)
21
22     cgi-bin/              # Compiled CGI files
23         guess_number.cgi
24         tictactoe_limited.cgi
25         ...
26
27     data/                 # JSON files
28         users.json
29         sessions.json
30         admin.json
31         admin_logs.json
32
```

Listing 1: پوشه‌ها ساختار

۳.۳ جریان اجرا

جریان کلی از انتخاب بازی تا اجرا به شرح زیر است:

۱. کاربر صفحه اصلی را باز می‌کند (index.html)
۲. JavaScript کاروسل بازی‌ها را نمایش می‌دهد
۳. کاربر یک بازی را انتخاب می‌کند
۴. سیستم احراز هویت بررسی می‌شود (auth.js)
۵. موجودی سکه کاربر بررسی می‌شود
۶. در صورت کافی بودن، سکه کسر می‌شود
۷. یک iframe با URL مربوط به بازی ساخته می‌شود
۸. سرور Python درخواست را به CGI مربوطه ارسال می‌کند
۹. برنامه CGI اجرا شده و HTML تولید می‌کند
۱۰. مرورگر HTML را در iframe نمایش می‌دهد
۱۱. کاربر با بازی تعامل می‌کند و CGI مجدداً با state جدید اجرا می‌شود

۴ بازی‌های موجود

سیستم بازی‌لند دارای ۷ بازی است که هر کدام به صورت مستقل پیاده‌سازی شده‌اند.

۱.۴ حدس عدد (Guess Number)

توضیح: بازیکن باید عدد تصادفی بین ۱ تا ۱۰۰ را حدس بزند. سطوح دشواری:

- آسان: ۱۰ تلاش، راهنمایی دقیق
- متوسط: ۷ تلاش، راهنمایی کلی
- سخت: ۵ تلاش، راهنمایی مبهم

قیمت: ۳ سکه
پاداش: ۵ سکه

۲.۴ دوز محدود (Tic-Tac-Toe Limited)

توضیح: نسخه تغییر یافته دوز با محدودیت ۳ مهره برای هر بازیکن. قانون خاص: وقتی مهره چهارم گذاشته می‌شود، قدیمی‌ترین مهره به صورت FIFO حذف می‌شود.
منطق FIFO:

```
1 if (count(player_moves) >= 3) {  
2     remove_oldest_move(player_moves);  
3 }  
4 add_new_move(player_moves, move);  
5
```

Listing 2: محدود دوز در FIFO الگوریتم

قیمت: ۵ سکه
پاداش: ۸ سکه

۳.۴ سنگ کاغذ قیچی ++ (RPS 5-Move)

توضیح: نسخه ۵ حرکتی از سریال Big Bang Theory. حرکات:

- Rock (سنگ)
- Paper (کاغذ)
- Scissors (قیچی)
- Lizard (مارمولک)
- Spock (اسپاک)

قوانین:

- سنگ > قیچی، مارمولک
- کاغذ > سنگ، اسپاک
- قیچی > کاغذ، مارمولک
- مارمولک > کاغذ، اسپاک
- اسپاک > سنگ، قیچی

قیمت: ۴ سکه
پاداش: ۶ سکه

۴.۴ حلقه آویز (Hangman Battle)

توضیح: دو بازیکن هم‌زمان در دو بازی حلقه آویز شرکت می‌کنند. حالت‌ها:

● Player vs Player

● Player vs Bot

استراتژی Bot: بر اساس فرکانس حروف در زبان انگلیسی:

ETAOINSHRDL CUMWFGYPBVKJXQZ

قیمت: ۶ سکه
پاداش: ۱۰ سکه

۵.۴ فیبوناچی ۲۰۴۸

توضیح: نسخه تغییر یافته ۲۰۴۸ با قوانین دنباله فیبوناچی. قانون اصلی: فقط اعداد مجاور در دنباله فیبوناچی می‌توانند ترکیب شوند:

$$1 + 1 = 2, \quad 1 + 2 = 3, \quad 2 + 3 = 5, \quad 3 + 5 = 8, \quad \dots$$

اندازه‌های صفحه: ۴ × ۴، ۵ × ۵، ۶ × ۶

قیمت: ۷ سکه
پاداش: ۱۲ سکه

۶.۴ شطرنج تغییر یافته

توضیح: شطرنج با ۳ مهره جدید. مهره‌های جدید:

- اژدها (Dragon): جایگزین اسب. ترکیب حرکت اسب + شاه (۱۶ حرکت ممکن).
- دزد (Thief): جایگزین فیل. فیل با قابلیت پرش از یک مانع.
- گریفین (Gryphon): جایگزین رخ. یک خانه مورب + سپس حرکت رخ.

تایمر: هر بازیکن ۱۰ دقیقه
 قیمت: ۱۰ سکه
 پاداش: ۲۰ سکه

۷.۴ چکرز (Checkers)

توضیح: بازی چکرز کلاسیک با قوانین استاندارد. قوانین اصلی:

- حرکت مهره: یک خانه مورب به جلو
- پرش: خوردن مهره حریف
- پرش زنجیره‌ای: اگر بعد از پرش، پرش دیگری ممکن باشد
- King شدن: رسیدن به انتهای صفحه

قیمت: ۸ سکه
 پاداش: ۱۵ سکه

۵ جزئیات پیاده‌سازی

۱.۵ Backend: توابع کمکی CGI

تمام بازی‌های CGI از توابع مشترکی استفاده می‌کنند که در `c_utils.h/c` تعریف شده‌اند.

۱.۱.۵ تابع `print_header()`

این تابع HTTP header لازم برای CGI را چاپ می‌کند:

```
1 void print_header() {
2     printf("Content-Type: text/html; charset=UTF-8\n\n");
3 }
4
```

Listing 3: تابع `print_header`

۲.۱.۵ تابع `get_param()`

این تابع مقدار یک پارامتر از `QUERY_STRING` را استخراج می‌کند:

```

1 char* get_param(const char* name) {
2     char *query = getenv("QUERY_STRING");
3     if (!query) return NULL;
4
5     // Parse query string
6     char *token = strtok(query, "&");
7     while (token) {
8         char *eq = strchr(token, '=');
9         if (eq) {
10            *eq = '\0';
11            if (strcmp(token, name) == 0) {
12                char *value = eq + 1;
13                char *decoded = malloc(MAX_PARAM_LEN);
14                url_decode(decoded, value);
15                return decoded;
16            }
17        }
18        token = strtok(NULL, "&");
19    }
20    return NULL;
21 }
22
```

Listing 4: تابع `get_param`۳.۱.۵ تابع `url_decode()`

این تابع URL encoding را به رشته عادی تبدیل می‌کند:

```

1 void url_decode(char* dst, const char* src) {
2     while (*src) {
3         if (*src == '%') {
4             int value;
5             sscanf(src + 1, "%2x", &value);
6             *dst++ = (char)value;
7             src += 3;
8         } else if (*src == '+') {
9             *dst++ = ' ';
10            src++;
11        } else {
12            *dst++ = *src++;
13        }
14    }
15    *dst = '\0';
16 }
17
```

Listing 5: تابع `url_decode`

۲.۵ Frontend معماری JavaScript

رابط کاربری با استفاده از Vanilla JavaScript پیاده‌سازی شده است.

۱.۲.۵ کاروسل بازی‌ها

کاروسل بازی‌ها با استفاده از transform: translateX() پیاده‌سازی شده:

```

1  function updateCarousel() {
2      const itemWidth = 120; // px
3      const gap = 20;        // px
4      const offset = -(currentIndex * (itemWidth + gap));
5      carousel.style.transform = `translateX(${offset}px)`;
6      updateActiveGame();
7  }
8
9  function nextGame() {
10     currentIndex = (currentIndex + 1) % gameItems.length;
11     updateCarousel();
12 }
13
14 function prevGame() {
15     currentIndex = (currentIndex - 1 + gameItems.length)
16     % gameItems.length;
17     updateCarousel();
18 }
19

```

Listing 6: بازی‌ها کاروسل

فرمول ریاضی موقعیت کاروسل:

$$\text{Position} = -(\text{index} \times (\text{itemWidth} + \text{gap}))$$

۲.۲.۵ ماژول احراز هویت

ماژول auth.js با استفاده از الگوی IIFE پیاده‌سازی شده:

```

1  const Auth = (function() {
2      const API_BASE = '/cgi-bin/auth_api.cgi';
3
4      // Private functions
5      function saveSession(user) {
6          localStorage.setItem('user', JSON.stringify(user));
7          localStorage.setItem('sessionToken', user.sessionToken)
8      };
9
10     // Public API
11     return {
12         login: async function(username, password) {
13             const response = await fetch(`${API_BASE}?action=
login`, {
14                 method: 'POST',
15                 body: JSON.stringify({ username, password })

```

```

16         });
17         const data = await response.json();
18         if (data.success) {
19             saveSession(data.user);
20         }
21         return data;
22     },
23
24     isLoggedIn: function() {
25         return localStorage.getItem('user') !== null;
26     },
27
28     getUser: function() {
29         const userStr = localStorage.getItem('user');
30         return userStr ? JSON.parse(userStr) : null;
31     }
32 };
33 })();
34

```

Listing 7: هویت احراز مازول

۳.۵ Frontend افکت‌های بصری CSS

۱.۳.۵ افکت نئونی (Neon Glow)

برای ایجاد افکت نئونی، از چندین لایه text-shadow استفاده می‌شود:

```

1 .marquee {
2     color: var(--neon-green);
3     text-shadow:
4         0 0 5px #fff,
5         0 0 10px var(--neon-green),
6         0 0 20px var(--neon-green),
7         0 0 40px var(--neon-green);
8 }
9

```

Listing 8: نئونی افکت

۲.۳.۵ افکت سه‌بعدی (3D Perspective)

برای کنترل دک از rotateX و perspective استفاده شده:

```

1 .control-deck {
2     perspective: 500px;
3 }
4
5 .deck-surface {
6     transform: rotateX(20deg);
7     transform-style: preserve-3d;
8 }
9

```

Listing 9: سه‌بعدی افکت

۳.۳.۵ افکت CRT

برای شبیه‌سازی مانیتور CRT، از خطوط اسکن و شکل بیضوی استفاده شده:

```
1 .crt-screen {
2     border-radius: 80px / 30px;
3     position: relative;
4 }
5
6 .crt-screen::before {
7     content: '';
8     position: absolute;
9     top: 0;
10    left: 0;
11    width: 100%;
12    height: 100%;
13    background: repeating-linear-gradient(
14        0deg,
15        rgba(0, 0, 0, 0.15),
16        rgba(0, 0, 0, 0.15) 1px,
17        transparent 1px,
18        transparent 2px
19    );
20    pointer-events: none;
21 }
22
```

Listing 10: افکت CRT

۴.۵ لایه داده: JSON Files

اطلاعات کاربران، سشن‌ها، و لاگ‌ها در فایل‌های JSON ذخیره می‌شوند.

۱.۴.۵ ساختار users.json

```
1 {
2     "users": [
3     {
4         "username": "admin",
5         "password": "hashed_password",
6         "coins": 1000,
7         "role": "admin",
8         "created_at": "2025-12-26T10:30:00Z",
9         "last_login": "2025-12-26T15:20:00Z"
10    }
11    ]
12 }
13
```

Listing 11: ساختار users.json

۲.۴.۵ ساختار admin_logs.json


```
1      {
2          "logs": [
3              {
4                  "id": 1,
5                  "username": "ali",
6                  "action": "subtract",
7                  "amount": 5,
8                  "reason": "Tic-Tac-Toe Limited",
9                  "timestamp": "2025-12-26T15:10:00Z",
10                 "before": 50,
11                 "after": 45
12             }
13         ]
14     }
15
```

Listing 12: ساختار admin_logs.json

۶ تصمیمات طراحی

۱.۶ انتخاب C برای Backend

دلایل:

- عملکرد: کامپایل شده و سریع
- یادگیری: تجربه کار با memory management و pointers
- چالش: سخت‌تر از PHP یا Node.js
- سبک: بدون نیاز به runtime سنگین

معایب:

- Debug سخت‌تر
- مدیریت string پیچیده
- نیاز به manual memory management

۲.۶ انتخاب CGI به جای FastCGI

دلایل:

- سادگی: نیاز به setup کمتر
- آموزشی: درک بهتر از HTTP protocol
- مستقل بودن: هر بازی یک process مجزا

- کافی برای این پروژه: traffic بالا نداریم

محدودیت‌ها:

- هر درخواست یک process جدید
- Overhead بالا برای traffic زیاد
- مقیاس‌پذیری محدود

۳.۶ انتخاب Vanilla JavaScript

دلایل:

- یادگیری: درک DOM و events بدون abstraction
- سبک: بدون overhead کتابخانه‌ها
- کنترل کامل: هر خط کد خودمان است
- عملکرد: سریع‌تر برای پروژه کوچک

معایب:

- کد بیشتر برای state management
- بدون reactivity خودکار
- DOM manipulation دستی

۴.۶ انتخاب فایل‌های JSON به جای پایگاه داده

دلایل:

- Setup ساده (بدون MySQL یا PostgreSQL)
- خواندن/نوشتن آسان از C
- Portable (فقط یک پوشه data/)
- مناسب برای تعداد کم کاربر

محدودیت‌ها:

- مقیاس‌پذیری کم (برای $1000 >$ کاربر)
- بدون transaction (مشکل race condition)
- بدون indexing (جستجو $O(n)$)

۷ چالش‌ها و محدودیت‌ها

۱.۷ چالش‌های فنی

۱.۱.۷ مدیریت State در CGI

از آنجا که CGI stateless است، تمام state باید در URL باشد. این باعث محدودیت طول URL می‌شود. راه‌حل: استفاده از encoding کارآمد و حذف اطلاعات غیرضروری.

۲.۱.۷ Race Condition در نوشتن فایل‌های JSON

اگر دو CGI همزمان بخواهند در `users.json` بنویسند، ممکن است داده از دست برود. راه‌حل فعلی: نداریم (برای پروژه کوچک مشکل ایجاد نمی‌کند)

راه‌حل آینده: استفاده از file locking با `flock()`

```
1 #include <sys/file.h>
2
3 int fd = open("data/users.json", O_RDWR);
4 flock(fd, LOCK_EX); // exclusive lock
5 // ... read/write ...
6 flock(fd, LOCK_UN); // unlock
7 close(fd);
8
```

Listing 13: File locking

۲.۷ محدودیت‌های امنیتی

۱.۲.۷ رمزهای عبور Plain Text

مشکل: رمزهای عبور در `users.json` به صورت متن ساده ذخیره می‌شوند. راه‌حل آینده: استفاده از `bcrypt`

```
1 #include <bcrypt.h>
2
3 char hash[BCRYPT_HASHSIZE];
4 bcrypt_hashpw(password, hash);
5 // Verify password
6 if (bcrypt_checkpw(password, hash) == 0) {
7     // Correct
8 }
9
```

Listing 14: استفاده از bcrypt

۲.۲.۷ عدم HTTPS

مشکل: تمام ترافیک plain text است. راه‌حل آینده: استفاده از Let's Encrypt برای SSL certificate.

۳.۲.۷ عدم CSRF Protection

مشکل: حملات Cross-Site Request Forgery ممکن است. راه‌حل آینده: استفاده از CSRF token در فرم‌ها.

۸ تست و اعتبارسنجی

۱.۸ تست دستی

تمام بازی‌ها به صورت دستی روی مرورگرهای زیر تست شده‌اند:

● Mobile) & (Desktop Chrome 120+

● Mobile) & (Desktop Firefox 120+

● Edge 120+

۲.۸ تست CGI

برای تست مستقیم CGI بدون سرور:

```
1 # Test without parameters
2 ./cgi-bin/guess_number.cgi
3
4 # Test with parameters
5 QUERY_STRING="mode=start&diff=1" ./cgi-bin/guess_number.cgi
6
```

Listing 15: تست CGI

۳.۸ محدودیت‌های تست

● بدون unit test خودکار

● بدون integration test

● بدون load test

۹ کارهای آینده

۱.۹ بهبودهای فنی

۱.۱.۹ انتقال به پایگاه داده

جایگزینی فایل‌های JSON با SQLite یا PostgreSQL:

```
1 CREATE TABLE users (  
2   id INTEGER PRIMARY KEY AUTOINCREMENT,  
3   username TEXT UNIQUE NOT NULL,  
4   password TEXT NOT NULL,  
5   coins INTEGER DEFAULT 20,  
6   role TEXT DEFAULT 'user',  
7   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8 );  
9  
10 CREATE TABLE sessions (  
11   id INTEGER PRIMARY KEY AUTOINCREMENT,  
12   session_token TEXT UNIQUE NOT NULL,  
13   user_id INTEGER NOT NULL,  
14   expires_at TIMESTAMP NOT NULL,  
15   FOREIGN KEY (user_id) REFERENCES users(id)  
16 );  
17
```

Listing 16: Schema پیشنهادی

۲.۱.۹ استفاده از WebSocket

برای بازی‌های real-time multiplayer:

- Chess با تایمر زنده
- Tic-Tac-Toe آنلاین
- سیستم چت

۳.۱.۹ امنیت

- Hash کردن رمزهای عبور با bcrypt
- پیاده‌سازی HTTPS
- CSRF protection
- Rate limiting
- Input validation بهتر

۲.۹ فیچرهای جدید

۱.۲.۹ بازی‌های جدید

- Snake
- Tetris
- real-time (multiplayer Pong)
- Pac-Man

۲.۲.۹ Leaderboard

جدول امتیازات برتر با جزئیات:

- امتیاز کل
- تعداد بازی‌های برد/باخت
- میانگین زمان بازی
- رتبه در هر بازی جداگانه

۳.۲.۹ سیستم دستاوردها (Achievements)

- برنده شدن ۱۰ بازی پیاپی
- بازی کردن تمام بازی‌ها
- رسیدن به ۱۰۰۰ سکه
- برنده شدن بدون باخت در روز

۳.۹ بهبودهای UI/UX

- افزودن Sound Effects
- انیمیشن‌های smooth تر
- حالت Dark/Light Theme
- بهینه‌سازی بیشتر برای موبایل
- Tutorial تعاملی برای کاربران جدید

۱۰ نتیجه‌گیری

۱.۱۰ دستاوردها

در این پروژه موفق به دستیابی به اهداف زیر شدیم:

۱. پیاده‌سازی کامل سیستم آرکید: شامل ۷ بازی، سیستم احراز هویت، سیستم سکه، و پنل مدیریتی.
۲. استفاده از معماری CGI: درک عمیق از نحوه کار وب اپلیکیشن‌های قدیمی و مفاهیم بنیادی HTTP.
۳. طراحی Stateless: پیاده‌سازی بازی‌هایی که تمام state در URL است و هیچ اطلاعاتی روی سرور ذخیره نمی‌شود.

۴. رابط کاربری منحصر به فرد: ایجاد یک تجربه بصری نوستالژیک با افکتهای سه‌بعدی، نئونی، و انیمیشن‌های پیچیده.
۵. توسعه بدون Framework: درک بهتر DOM، event handling، و state management بدون استفاده از کتابخانه‌های سنگین.
۶. مستندسازی جامع: ایجاد بیش از ۳۰۰۰ خط کامنت فارسی در کد و مستندات کامل README.

۲.۱۰ درس‌های آموخته شده

۱.۲.۱۰ معماری CGI

معماری CGI اگرچه قدیمی است، اما مفاهیم بنیادی خوبی را آموزش می‌دهد:

- نحوه کار HTTP
- جریان request/response
- مدیریت state بدون سرور

برای پروژه‌های کوچک و آموزشی مناسب است، اما برای production و traffic بالا به روش‌های جدیدتری مانند REST API یا GraphQL نیاز است.

۲.۲.۱۰ State Management

مدیریت state در سیستم stateless چالش بزرگی است. استفاده از URL parameters روش ساده اما محدودی است. برای بازی‌های پیچیده‌تر، نیاز به روش‌های بهتری مانند session storage یا database است.

۳.۲.۱۰ امنیت

امنیت یک فکر بعدی (afterthought) نباید باشد. از ابتدای پروژه باید به مسائل امنیتی توجه شود:

- Hash کردن رمزها
- استفاده از HTTPS
- Input validation
- CSRF protection

۳.۱۰ کاربردهای آموزشی

این پروژه برای موارد زیر قابل استفاده است:

- آموزش Web Development: مفاهیم بنیادی HTTP، CGI، DOM

- آموزش برنامه‌نویسی C: کار با string، pointer، file I/O
- آموزش JavaScript: Event handling، DOM manipulation، Fetch API
- آموزش CSS: Animations، Transforms، Grid، Flexbox
- طراحی سیستم: معماری stateless، separation of concerns

۴.۱۰ جمع‌بندی نهایی

سیستم بازی‌لند نشان می‌دهد که چگونه می‌توان با استفاده از تکنولوژی‌های ساده و بنیادی، یک سیستم کامل و کاربردی ساخت. اگرچه این سیستم برای محیط production آماده نیست، اما به عنوان یک پروژه آموزشی و نمونه کد، ارزش زیادی دارد. تجربه ساخت این پروژه نشان داد که:

- درک مفاهیم بنیادی مهم‌تر از استفاده از جدیدترین تکنولوژی‌ها است
- سادگی در طراحی، پیچیدگی در پیاده‌سازی را کاهش می‌دهد
- مستندسازی خوب، حفظ و توسعه پروژه را آسان‌تر می‌کند
- هر تصمیم طراحی، trade-off هایی دارد که باید آگاهانه انتخاب شوند

تشکر و قدردانی

از تمام منابع و ابزارهای Open Source که در این پروژه استفاده شده، تشکر می‌کنم:

- Font Awesome برای آیکون‌ها
- Particles.js برای افکت پس‌زمینه
- Vazirmatn Font برای فونت فارسی
- Python HTTP Server برای سرویس CGI

مراجع

- [۱] Robinson, D., & Coar, K. (2004). The Common Gateway Interface (CGI) Version 1.1. RFC 3875.
- [۲] Fielding, R., et al. (1999). Hypertext Transfer Protocol – HTTP/1.1. RFC 2616.
- [۳] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

<https://owasp.org/> OWASP Foundation. (2021). OWASP Top Ten Project. [۴]
[www-project-top-ten/](https://owasp.org/www-project-top-ten/)

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide> Mozilla Developer Network. (2025). JavaScript Guide. [۵]

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/> CSS-Tricks. (2025). A Complete Guide to Flexbox. [۶]

Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall. [۷]

Schell, J. (2014). The Art of Game Design: A Book of Lenses (2nd ed.). CRC Press. [۸]

آ نصب و راه‌اندازی

۱.آ پیش‌نیازها

```
1 # Install GCC
2 sudo apt-get install gcc
3
4 # Install Make
5 sudo apt-get install make
6
7 # Install Python 3
8 sudo apt-get install python3
9
```

۲.آ کامپایل

```
1 # Compile all games
2 make all
3
4 # Or compile a specific game
5 make guess_number.cgi
6
```

۳.آ اجرا

```
1 # Grant execution permissions to CGIs
2 chmod +x cgi-bin/*.cgi
3
4 # Start server
5 python3 -m http.server --cgi 8080
6
7 # Open in browser
8 # http://localhost:8080
9
```